The Boundary Element Method Applied to Laplace's Equation in 2D

Computational Electromagnetics - 2023/24

March 28, 2024

The Laplace equation $\nabla^2 u = 0$ models a wide variety of real-world problems, but with the exception of few relatively simple cases it is very hard to solve it exactly. A numerical solution on the other hand can be achieved by employing the Boundary Element Method (BEM), which transforms the differential operator in an integral calculated on boundary Γ of the domain Ω in which the problem is set, and finds an approximate solution of such a boundary integral by means of a discretization process [1]. This translates into the decomposition of Γ into a finite number of curves, each of which is parametrized with respect to a certain real parameter [2].
Two approximations are carried out: one is about the boundary's geometry, where the above-mentioned curves are approximated with the so-called elements, while the other one has to do with the variation of the boundary quantities over the elements. Depending on the degree of the polynomials adopted for such approximations, the formulation can be classified as sub-parametric, iso-parametric or super-parametric. The finitely many parameters determining the approximated solution are computed from finitely many linear equations. In the **collocation method** one chooses a suitable set of *collocation points* $\{p_i \mid i = 1, ..., N\} \subset \Gamma$ and requires that the boundary integral equation is satisfied in all the p_i.

1 Collocation method

One can distinguish, for each element, the end points and the nodal points. The first ones define the element's position in the xy-plane, whereas the latter are the points at which values of the boundary quantities are assigned [3].

The discretized form of the boundary integral equation, assumed N elements numbered in the counter-clockwise sense, is expressed for a given point p_i on Γ as

$$\epsilon(p_i)u(p_i) = -\sum_{j=1}^{N} \int_{\Gamma_j} \left[v(p_i, q) \frac{\partial u(q)}{\partial n_q} - u(q) \frac{\partial v(p_i, q)}{\partial n_q} \right] \mathrm{d}s_q \ , \tag{1}$$

where Γ_j is the j-th element over which integration is carried out, p_i is the reference/source nodal point, $\epsilon(p_i) = \alpha_i/2\pi$ and v is Green's function in 2D.

It is possible to re-write (1) in a different form,

$$\epsilon^i u^i = -\sum_{j=1}^{N} \int_{\Gamma_j} v u_n \, \mathrm{d}s + \sum_{j=1}^{N} \int_{\Gamma_j} u v_n \, \mathrm{d}s \ . \tag{2}$$

The last equation is applied consecutively for all the nodes p_i, yielding a system of linear algebraic equations.

1.1 Constant BEM

In case of constant boundary elements, the boundary element is approximated by a straight line that connects its two extreme points. The node is placed at the mid-point, while the boundary quantity is assumed to be constant along the element and equal to its value at the nodal point. Since the geometry is represented with a polynomial of higher order than the one used to approximate the boundary quantity, it is said to be depicted super-parametrically [4].

Given these considerations, (2) becomes

$$-\frac{1}{2}u^i + \sum_{j=1}^{N} \left(\int_{\Gamma_j} v_n \, \mathrm{d}s \right) u^j = \sum_{j=1}^{N} \left(\int_{\Gamma_j} v \, \mathrm{d}s \right) u_n^j \ . \tag{3}$$

This result arises due to the fact that it is always $\alpha_i = \pi$ for the i-th boundary node placed at the element's mid-point, while quantities u and u_n are constant over the j-th element.

The integrals involved in the above equation relate node p_i, where the fundamental solution is applied, to node p_j, where the boundary condition is defined; they are often referred to as <u>influence coefficients</u> and they are defined as

$$\hat{H}_{ij} = \int_{\Gamma_j} \frac{\partial v(p_i, q)}{\partial n_q} \, \mathrm{d}s_q \ ,$$
$$G_{ij} = \int_{\Gamma_j} v(p_i, q) \, \mathrm{d}s_q \ ,$$

where the reference point p_i remains constant while the integration point q varies over the j-th element.

Introducing the influence coefficients in (3), the solution becomes

$$-\frac{1}{2}u^i + \sum_{j=1}^{N} \hat{H}_{ij} u^j = \sum_{j=1}^{N} G_{ij} u_n^j \ . \tag{4}$$

Moreover, setting

$$H_{ij} = \hat{H}_{ij} - \frac{1}{2}\delta_{ij} \quad \begin{cases} \delta_{ij} = 0 & \text{for } i \neq j \\ \delta_{ij} = 1 & \text{for } i = j \end{cases} \quad , \tag{5}$$

results in the following liner system of equations

$$\sum_{j=1}^{N} H_{ij} u^j = \sum_{j=1}^{N} G_{ij} u_n^j \quad \Rightarrow \quad [H]\{u\} = [G]\{u_n\} \ . \tag{6}$$

1.2 Linear BEM

For linear elements, the boundary element is approximated again by a straight line connecting its end points. Two nodes are placed at the extreme points of the element and the boundary quantity is assumed to vary linearly between the nodal values. Since the geometry and the boundary quantity are approximated over the element by polynomials of the same degree, the former is said to be depicted iso-parametrically.

In order to establish the expression for the variation of the boundary quantity over the j-th element, it is convenient to introduce a local coordinate system, as shown in Fig. 1.

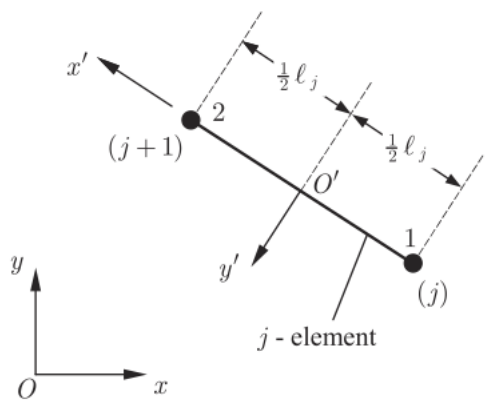

Figure 1: Global and local system of axes for the element j.

In the local system of axes

$$-\frac{l_j}{2} \leq x' \leq +\frac{l_j}{2} \quad , \quad y' = 0 \ ,$$

whereas in the global system of axes

$$x = \frac{x_{j+1} + x_j}{2} + \frac{x_{j+1} - x_j}{l_j} x' \ ,$$
$$y = \frac{y_{j+1} + y_j}{2} + \frac{y_{j+1} - y_j}{l_j} x' \ .$$

The interval $\left[-l_j/2, +l_j/2\right]$ is then normalized by the introduction of a parameter ξ defined as

$$\xi = \frac{x'}{l_j/2} \ ,$$

and it finally results the expression of the global coordinates as a function of ξ

$$\begin{aligned} x &= \frac{x_{j+1} + x_j}{2} + \frac{x_{j+1} - x_j}{2}\xi \\ y &= \frac{y_{j+1} + y_j}{2} + \frac{y_{j+1} - y_j}{2}\xi \end{aligned} \quad \Rightarrow \quad \begin{aligned} x(\xi) &= \frac{1}{2}(1-\xi)x_j + \frac{1}{2}(1+\xi)x_{j+1} \\ y(\xi) &= \frac{1}{2}(1-\xi)y_j + \frac{1}{2}(1+\xi)y_{j+1} \end{aligned} \ ,$$

where

$$-1 \leq \xi \leq +1 \ .$$

Since the linear element is defined iso-parametrically, the same considerations hold for the variation of the boundary quantity, leading to a general equation

$$f(\xi) = \psi_1(\xi) f_1 + \psi_2(\xi) f_2 \ , \tag{7}$$

where f_1 and f_2 are the values of function $f(x, y)$ at nodes 1 (i.e. j) and 2 (i.e. $j+1$), and $f(\xi)$ represents any of the functions $x(\xi)$, $y(\xi)$, $u(\xi)$ or $u_n(\xi)$. The remaining terms are referred to as <u>linear shape functions</u>

$$\begin{aligned} \psi_1(\xi) &= \frac{1}{2}(1-\xi) \ , \\ \psi_2(\xi) &= \frac{1}{2}(1+\xi) \ . \end{aligned} \tag{8}$$

shape_fun

```
% Calculates the value of the shape function in the given point t, which
% has to be in [-1, 1].
%
%    met    Geometric element type (1 for linear elements, 2 for quadratic).
%     c     Shape function type (2 is allowed only with quadratic elements).
%     t     The point in [-1, 1] where the function has to be evaluated.

function [sf] = shape_fun(met, c, t)

    if met == 1
```

```
        % Linear elements

        if c == 1
            sf = 0.5 * (1-t);
        elseif c == 3
            sf = 0.5 * (1+t);
        else
            disp('Shape function error: c cannot be 2')
        end
```

Using the linear approximation (7) for the boundary quantities, the line integrals appearing in (2) become

$$\int_{\Gamma_j} v u_n \, \mathrm{d}s = \int_{-1}^{+1} v \left[\psi_1(\xi) u_n^1 + \psi_2(\xi) u_n^2\right] \frac{l_j}{2} \, \mathrm{d}\xi = g_1^{ij} u_n^1 + g_2^{ij} u_n^2 \ , \tag{9}$$

$$\int_{\Gamma_j} u v_n \, \mathrm{d}s = \int_{-1}^{+1} v_n \left[\psi_1(\xi) u^1 + \psi_2(\xi) u^2\right] \frac{l_j}{2} \, \mathrm{d}\xi = h_1^{ij} u^1 + h_2^{ij} u^2 \ , \tag{10}$$

where the coefficients are

$$g_{1/2}^{ij} = \frac{l_j}{2} \int_{-1}^{+1} v(\xi) \psi_{1/2}(\xi) \, \mathrm{d}\xi \ , \tag{11}$$

$$h_{1/2}^{ij} = \frac{l_j}{2} \int_{-1}^{+1} v_n(\xi) \psi_{1/2}(\xi) \, \mathrm{d}\xi \ , \tag{12}$$

while Green's function v and its normal derivative v_n are

$$v(\xi) = \frac{1}{2\pi} \ln r(\xi) \ ,$$

$$v_n(\xi) = \frac{\partial v(\xi)}{\partial n} = \frac{1}{2\pi} \frac{\cos \phi(\xi)}{r(\xi)} \ ,$$

considering $r = \sqrt{\left[x(\xi)^2 - x_i\right]^2 + \left[y(\xi)^2 - y_i\right]^2}$ as the distance between the source point (x_i, y_i) and the boundary integration point $\big(x(\xi), y(\xi)\big)$.

find_N

```
% Calculates the normal versor N(t) = (nx(t), ny(t)) needed to evaluate the
% kernel.
%
%       met       Geometric element type (1 for linear elements, 2 for
%                 quadratic).
%
%    (x1, y1) (x2, y2) (x3, y2)
%                 The boundary points that identify the element. In case of
%                 linear elements point (x2, y2) is the middle point between
%                 (x1, y1) and (x3, y3).
%
%        t        The point in [-1, 1] where the function has to be
%                 evaluated.
```

```
function [nx, ny] = find_N(met, x1, y1, x2, y2, x3, y3, t)

    % Infinitesimal increment for the numerical
    % evaluation of the normal direction.
    dt = 1e-5;

    if met == 1

        % Linear elements

        xt_p = shape_fun(met, 1, t+dt) * x1 + ...
               shape_fun(met, 3, t+dt) * x3;

        yt_p = shape_fun(met, 1, t+dt) * y1 + ...
               shape_fun(met, 3, t+dt) * y3;

        xt_m = shape_fun(met, 1, t-dt) * x1 + ...
               shape_fun(met, 3, t-dt) * x3;

        yt_m = shape_fun(met, 1, t-dt) * y1 + ...
               shape_fun(met, 3, t-dt) * y3;

    ls = sqrt( (xt_p - xt_m).^2 + (yt_p - yt_m).^2 );

    nx = ( yt_p - yt_m ) ./ ls;
    ny = ( xt_m - xt_p ) ./ ls;
```

find_R

```
% Calculates the radius R(t) = (rx(t), ry(t)) needed to evaluate the
% kernel.
%
%      met       Geometric element type (1 for linear elements, 2 for
%                quadratic).
%
%    (xi, yi)    The reference point.
%
%    (x1, y1) (x2, y2) (x3, y2)
%                The boundary points that identify the element. In case of
%                linear elements point (x2, y2) is the middle point between
%                (x1, y1) and (x3, y3).
%
%       t        The point in [-1, 1] where the function has to be
%                evaluated.

function [rx, ry] = find_R(met, xi, yi, x1, y1, x2, y2, x3, y3, t)

    if met==1

        % Linear elements

        xt = shape_fun(met, 1, t) * x1 + ...
             shape_fun(met, 3, t) * x3;

        yt = shape_fun(met, 1, t) * y1 + ...
             shape_fun(met, 3, t) * y3;

        disp('Shape function error: supported element types are 1 or 2.');
```

Substituting (9) and (10) back into (2), the latter yields

$$-\epsilon^i u^i + \sum_{j=1}^{N} \hat{H}_{ij} u^j = \sum_{j=1}^{N} G_{ij} u_n^j \ , \tag{13}$$

where

$$\hat{H}_{ij} = \begin{cases} h_1^{i1} + h_2^{iN} & \text{for} \quad j = 1 \\ h_1^{ij} + h_2^{i(j-1)} & \text{for} \quad j = 2, 3, \ldots, N \end{cases} \ ,$$

$$G_{ij} = \begin{cases} g_1^{i1} + g_2^{iN} & \text{for} \quad j = 1 \\ g_1^{ij} + g_2^{i(j-1)} & \text{for} \quad j = 2, 3, \ldots, N \end{cases} \ .$$

Equation (13) may be written in matrix form as

$$[H]\{u\} = [G]\{u_n\} \ , \tag{14}$$

having defined

$$[H] = -[\epsilon] + \left[\hat{H}\right] \ ,$$

where $[\epsilon]$ is a diagonal matrix of coefficients $\epsilon^i = \alpha_i / 2\pi$ (α_i is the angle between elements i and $i-1$).

find_BEM_unknowns

```
% Calculates the vector z of BEM unknowns on the nodes (xn,yn).
%
%    met    Geometric element type (1 for linear elements, 2 for quadratic).
%    xn     Nodes' x-coordinates.
%    yn     Nodes' y-coordinates.
%    bc     Boundary conditions at each node (0 for Dirichlet, 1 for
%           Neumann).
%    bv     Boundary values for the assigned boundary conditions.

function [z] = find_BEM_unknowns(met, xn, yn, bc, bv)

    xn(end+1) = xn(1);
    yn(end+1) = yn(1);

    % Total number of BEM nodes
    I = length(xn) - 1;

    % Construction of G and H matrices

    H = zeros(I, I);
    G = zeros(I, I);

    if met == 1

        % Linear elements

        xm = zeros(I, 1);
```

```
ym = zeros(I, 1);

for i = 1:I
    for j = 1:I

        % Mid-point coordinates necessary in findH().
        xm(j) = 0.5 * (xn(j) + xn(j+1));
        ym(j) = 0.5 * (yn(j) + yn(j+1));

        if j == 1 % First column (problem j-1 = I when j = 1)
            if j == i
                H(i,j) = 0;
                G(i,j) =                                                ...
                    findG_lin_s(xn(1), yn(1), xn(2), yn(2))             ...
                    +                                                   ...
                    findG_lin_s(xn(I), yn(I), xn(1), yn(1));
            else
                H(i,j) =                                                ...
                    findH(met, 1, xn(i), yn(i), xn(1), yn(1),           ...
                                  xm(1), ym(1), xn(2), yn(2)  )         ...
                    +                                                   ...
                    findH(met, 3, xn(i), yn(i), xn(I), yn(I),           ...
                                  xm(I), ym(I), xn(1), yn(1)  );
                G(i,j) =                                                ...
                    findG(met, 1, xn(i), yn(i), xn(1), yn(1),           ...
                                  xm(1), ym(1), xn(2), yn(2)  )         ...
                    +                                                   ...
                    findG(met, 3, xn(i), yn(i), xn(I), yn(I),           ...
                                  xm(I), ym(I), xn(1), yn(1)  );
            end
        else % Other columns
            if j == i
                H(i,j) = 0;
                G(i,j) =                                                ...
                    findG_lin_s( xn( j ), yn( j ),                      ...
                                 xn(j+1), yn(j+1)  )                    ...
                    +                                                   ...
                    findG_lin_s( xn(j-1), yn(j-1),                      ...
                                 xn( j ), yn( j )  );
            else
                H(i,j) =                                                ...
                    findH(met, 1, xn( i ), yn( i ),                     ...
                                  xn( j ), yn( j ),                     ...
                                  xm( j ), ym ( j ),                    ...
                                  xn(j+1), yn(j+1)  )                   ...
                    +                                                   ...
                    findH(met, 3, xn( i ), yn( i ),                     ...
                                  xn(j-1), yn(j-1),                     ...
                                  xm(j-1), ym(j-1),                     ...
                                  xn( j ), yn( j )  );
                G(i,j) =                                                ...
                    findG(met, 1, xn( i ), yn( i ),                     ...
                                  xn( j ), yn( j ),                     ...
                                  xm( j ), ym( j ),                     ...
                                  xn(j+1), yn(j+1)  )                   ...
                    +                                                   ...
                    findG(met, 3, xn( i ), yn( i ),                     ...
                                  xn(j-1), yn(j-1),                     ...
                                  xm(j-1), ym(j-1),                     ...
                                  xn( j ), yn( j )  );
            end
        end
```

```
        end

        % Indirect evaluation of the diagonal influence coefficients
        H(i,i) = -sum(H(i,:));
    end
```

1.3 Quadratic BEM

Constant and linear elements cannot approximate with sufficient accuracy the geometry of curvilinear boundaries. In such cases it is recommended to use curvilinear elements, whose polynomials are of degree greater than one.

For quadratic elements the geometry is approximated by a parabolic arc, connecting the two end-points with the mid-one: each element then has three nodes, and the boundary quantity is assumed to vary quadratically among the nodal values. Since the geometry and the boundary quantity are approximated over the element by polynomials of the same degree, the former is said to be depicted iso-parametrically.

When a quadratic variation is assumed, the boundary quantities are expressed by a polynomial of the form

$$f(s) = \alpha_0 + \alpha_1 s + \alpha_2 s^2 \ . \tag{15}$$

The coordinates of point (x, y) lying on the boundary Γ are also functions of s, i.e. $x = x(s)$ and $y = y(s)$; consequently $v = v(s)$ and $v_n = v_n(s)$ too.

In order to perform line integrations, the interval is normalized with respect to the intrinsic coordinate ξ, leading to the transformation

$$I = \int_{\Gamma_j} w(s)\,\mathrm{d}s \quad \Rightarrow \quad I = \int_{-1}^{+1} w^*(\xi)|J(\xi)|\,\mathrm{d}\xi \ , \tag{16}$$

where $J(\xi)$ is the Jacobian of the transformation that maps the parabolic arc Γ_j in the xy-plane onto the straight line segment with $-1 \leq \xi \leq +1$ and $\eta = 0$ of the $\eta\xi$-plane (see Fig. 2)

$$\mathrm{d}s = \sqrt{\mathrm{d}x^2 + \mathrm{d}y^2} = \sqrt{\left[x'(\xi)\right]^2 + \left[y'(\xi)\right]^2}\,\mathrm{d}\xi \ .$$

The magnitude of the Jacobian then is

$$|J(\xi)| = \sqrt{\left[x'(\xi)\right]^2 + \left[y'(\xi)\right]^2} \ . \tag{17}$$

Fig. 2 shows graphically the normalization and the mapping of the quadratic element.

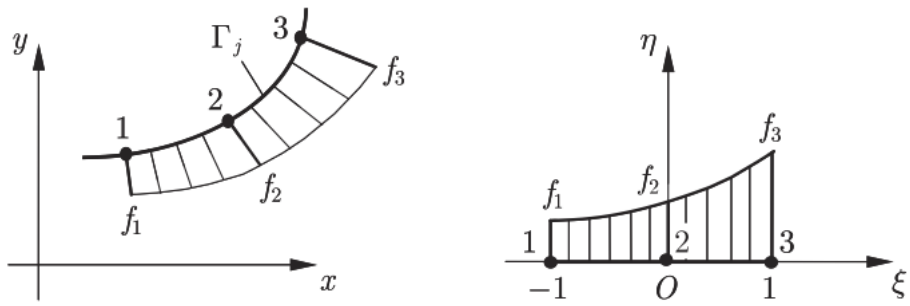

Figure 2: Quadratic element in global and local coordinate system.

The boundary quantity is approximated directly in the interval $[-1, +1]$ by a second order polynomial in ξ, according to (15)

$$f(\xi) = \alpha_0 + \alpha_1 \xi + \alpha_2 \xi^2 \ . \tag{18}$$

The coefficients α_0, α_1 and α_2 are determined from the requirement that the function takes the nodal values f_1, f_2 and f_3 at points $\xi = -1, 0, +1$, respectively:

$$\begin{cases} f(-1) = & f_1 = \alpha_0 - \alpha_1 + \alpha_2 \\ f(0) = & f_2 = \alpha_0 \\ f(+1) = & f_3 = \alpha_0 + \alpha_1 + \alpha_2 \end{cases} \Rightarrow \begin{cases} \alpha_0 = & f_2 \\ \alpha_1 = & \dfrac{f_3 - f_1}{2} \\ \alpha_2 = & \dfrac{f_1 - 2f_2 + f_3}{2} \end{cases} .$$

Introducing the resulting coefficients in (18), the expression of the boundary quantity in terms of the three nodal values becomes

$$f(\xi) = f_2 + \frac{f_3 - f_1}{2}\xi + \frac{f_1 - 2f_2 + f_3}{2}\xi^2 = \psi_1(\xi) f_1 + \psi_2(\xi) f_2 + \psi_3(\xi) f_3 \ ,$$

where the shape functions for the quadratic element are

$$\begin{aligned} \psi_1(\xi) &= -\frac{1}{2}\xi(1-\xi) \ , \\ \psi_2(\xi) &= (1-\xi)(1+\xi) \ , \\ \psi_3(\xi) &= +\frac{1}{2}\xi(1+\xi) \ . \end{aligned}$$

shape_fun

```
    elseif met == 2

        % Quadratic elements
```

```
        if c == 1
            sf = -0.5 * t .* (1-t);
        elseif c == 2
            sf = (1-t) .* (1+t);
        elseif c==3
            sf = +0.5 * t .* (1+t);
        end
```

Since the quadratic element is depicted iso-parametrically, the same considerations hold also for the geometry, which leads to

$$
\begin{aligned}
x(\xi) &= \psi_1(\xi)x_1 + \psi_2(\xi)x_2 + \psi_3(\xi)x_3 \ , \\
y(\xi) &= \psi_1(\xi)y_1 + \psi_2(\xi)y_2 + \psi_3(\xi)y_3 \ .
\end{aligned}
$$

Finally, the Jacobian can be expressed as

$$
|J(\xi)| = \sqrt{\left[x'(\xi)\right]^2 + \left[y'(\xi)\right]^2} = \sqrt{(b_1 + 2b_2\xi)^2 + (c_1 + 2c_2\xi)^2} \ , \tag{19}
$$

where

$$
\begin{aligned}
b_1 &= \frac{x_3 - x_1}{2}, \quad b_2 = \frac{x_1 - 2x_2 + x_3}{2} \\
c_1 &= \frac{y_3 - y_1}{2}, \quad c_2 = \frac{y_1 - 2y_2 + y_3}{2}
\end{aligned} \ .
$$

abs_J

```
% Calculates the jacobian for a given element (the element type is
% automatically recognized by the given point-coordinates).
%
%        t       The point in [-1, 1] where the function has to be
%                evaluated.
%
%    (x1, y1) (x2, y2) (x3, y2)
%                The boundary points that identify the element. In case of
%                linear elements point (x2, y2) is the middle point between
%                (x1, y1) and (x3, y3).

function [J] = abs_J(x1, y1, x2, y2, x3, y3, t)

    b1 = (x3 - x1) / 2;
    b2 = (x1 - 2*x2 + x3) / 2;
    c1 = (y3 - y1) / 2;
    c2 = (y1 - 2*y2 + y3) / 2;

    J = sqrt((b1 + 2*b2*t).^2 + (c1 + 2*c2*t).^2);

end
```

Taking into account the quadratic variation of the boundary quantities on the elements, the line integrals of (2) yields

$$
\begin{aligned}
\int_{\Gamma_j} v u_n \, \mathrm{d}s &= \int_{-1}^{+1} v \left[\psi_1(\xi)u_n^1 + \psi_2(\xi)u_n^2 + \psi_3(\xi)u_n^3\right] \frac{l_j}{2} \, \mathrm{d}\xi \\
&= g_1^{ij} u_n^1 + g_2^{ij} u_n^2 \ ,
\end{aligned} \tag{20}
$$

$$
\begin{aligned}
\int_{\Gamma_j} u v_n \, \mathrm{d}s &= \int_{-1}^{+1} v_n \left[\psi_1(\xi) u^1 + \psi_2(\xi) u^2 + \psi_3(\xi) u^3 \right] \frac{l_j}{2} \, \mathrm{d}\xi \\
&= h_1^{ij} u^1 + h_2^{ij} u^2 \ ,
\end{aligned} \tag{21}
$$

where, for $\alpha = 1, 2, 3$

$$
\begin{aligned}
g_\alpha^{ij} &= \int_{\Gamma_j} v \psi_\alpha \, \mathrm{d}s \ , \\
h_\alpha^{ij} &= \int_{\Gamma_j} \psi_\alpha v_n \, \mathrm{d}s \ .
\end{aligned}
$$

To evaluate such integrals we express their integrands in terms of ξ

$$
\begin{aligned}
g_\alpha^{ij} &= \int_{-1}^{+1} \psi_\alpha(\xi) \frac{\ln r(\xi)}{2\pi} |J(\xi)| \, \mathrm{d}\xi \ , \\
h_\alpha^{ij} &= \int_{-1}^{+1} \psi_\alpha(\xi) \frac{\cos \phi(\xi)}{2\pi r(\xi)} |J(\xi)| \, \mathrm{d}\xi \ .
\end{aligned}
$$

find_N

```
elseif met == 2

    % Quadratic elements

    xt_p = shape_fun(met, 1, t+dt) * x1 + ...
           shape_fun(met, 2, t+dt) * x2 + ...
           shape_fun(met, 3, t+dt) * x3;

    yt_p = shape_fun(met, 1, t+dt) * y1 + ...
           shape_fun(met, 2, t+dt) * y2 + ...
           shape_fun(met, 3, t+dt) * y3;

    xt_m = shape_fun(met, 1, t-dt) * x1 + ...
           shape_fun(met, 2, t-dt) * x2 + ...
           shape_fun(met, 3, t-dt) * x3;

    yt_m = shape_fun(met, 1, t-dt) * y1 + ...
           shape_fun(met, 2, t-dt) * y2 + ...
           shape_fun(met, 3, t-dt) * y3;

ls = sqrt( (xt_p - xt_m).^2 + (yt_p - yt_m).^2 );

nx = ( yt_p - yt_m ) ./ ls;
ny = ( xt_m - xt_p ) ./ ls;
```

find_R

```
elseif met == 2

    % Quadratic elements

    xt = shape_fun(met, 1, t) * x1 + ...
```

```
            shape_fun(met, 2, t) * x2 + ...
            shape_fun(met, 3, t) * x3;

        yt = shape_fun(met, 1, t) * y1 + ...
            shape_fun(met, 2, t) * y2 + ...
            shape_fun(met, 3, t) * y3;

    rx = xt - xi;
    ry = yt - yi;
```

The boundary integral equation, for quadratic elements, is discretized as

$$\epsilon^i u^i + \sum_{j=1}^{E} \sum_{\alpha=1}^{3} h_\alpha^{ij} u_\alpha^j = \sum_{j=1}^{E} \sum_{\alpha=1}^{3} g_\alpha^{ij} u_{n\alpha}^j \ , \tag{22}$$

where E is the total number of elements, i refers to the source point and j to the boundary element. Since for quadratic elements the number of nodes per element is 3, the closed boundary is characterized by $N = 2E$ nodes [5].

From the last statement important considerations follow. Index j, which has been always associated to the j-th element, must now be considered as the index referring to the node q where the boundary condition is prescribed. In this way, even for quadratic elements, i and j are respectively the row and column counters, and the matrices' size is $N \times N$. This results in the general linear system

$$[H]\{u\} = [G]\{u_n\} \ , \tag{23}$$

where

$$\hat{H}_{ij} = \begin{cases} h_1^{i1} + h_3^{iN} & \text{for } j = 1 \\ h_2^{ij} & \text{for j even } (j = 2, 4, \ldots, N) \\ h_1^{ij} + h_3^{i(j-1)} & \text{for j odd } (j = 3, 5, \ldots, N-1) \end{cases} \ , \tag{24}$$

$$G_{ij} = \begin{cases} g_1^{i1} + g_3^{iN} & \text{for } j = 1 \\ g_2^{ij} & \text{for j even } (j = 2, 4, \ldots, N) \\ g_1^{ij} + g_3^{i(j-1)} & \text{for j odd } (j = 3, 5, \ldots, N-1) \end{cases} \ . \tag{25}$$

As a result, for quadratic elements the coefficients at the element end points have two terms, related to the forward and backward incidence, while those at the element center points have only one term. As already mentioned, such influence coefficients relate node p_i, where the boundary solution is computed, to node p_j, where the boundary condition is defined.

find_BEM_unkowns

```
    elseif met == 2

        % Quadratic elements

        for i = 1:I
            for j = 1:I

                if j == 1 % First column (problem j-1 = I when j = 1)
                    if j == i
                        H(i,j) = 0;
                        G(i,j) =                                        ...
                            findG_par_s(1, xn( 1 ), yn( 1 ),            ...
                                           xn( 1 ), yn( 1 ),            ...
                                           xn( 2 ), yn( 2 ),            ...
                                           xn( 3 ), yn( 3 ),            ...
                                           -1 )                         ...
                            +                                           ...
                            findG_par_s(3, xn( 1 ), yn( 1 ),            ...
                                           xn(I-1), yn(I-1),            ...
                                           xn( I ), yn( I ),            ...
                                           xn( 1 ), yn( 1 ),            ...
                                           +1 );
                    else
                        H(i,j) =                                        ...
                            findH(met, 1, xn( i ), yn( i ),             ...
                                          xn( 1 ), yn( 1 ),             ...
                                          xn( 2 ), yn( 2 ),             ...
                                          xn( 3 ), yn( 3 )  )           ...
                            +                                           ...
                            findH(met, 3, xn( i ), yn( i ),             ...
                                          xn(I-1), yn(I-1),             ...
                                          xn( I ), yn( I ),             ...
                                          xn( 1 ), yn( 1 )  );
                        G(i,j) =                                        ...
                            findG(met, 1, xn( i ), yn( i ),             ...
                                          xn( 1 ), yn( 1 ),             ...
                                          xn( 2 ), yn( 2 ),             ...
                                          xn( 3 ), yn( 3 )  )           ...
                            +                                           ...
                            findG(met, 3, xn( i ), yn( i ),             ...
                                          xn(I-1), yn(I-1),             ...
                                          xn( I ), yn( I ),             ...
                                          xn( 1 ), yn( 1 )  );
                    end
                elseif mod(j,2) == 0 % Even columns
                    if j == i
                        H(i,j) = 0;
                        G(i,j) =                                        ...
                            findG_par_s(2, xn( i ), yn( i ),            ...
                                           xn(j-1), yn(j-1),            ...
                                           xn( j ), yn( j ),            ...
                                           xn(j+1), yn(j+1),            ...
                                           0 );
                    else
                        H(i,j) =                                        ...
                            findH(met, 2, xn( i ), yn( i ),             ...
                                          xn(j-1), yn(j-1),             ...
                                          xn( j ), yn( j ),             ...
                                          xn(j+1), yn(j+1)  );
                        G(i,j) =                                        ...
```

```
                    findG(met, 2, xn( i ), yn( i ),                 ...
                                  xn(j-1), yn(j-1),                 ...
                                  xn( j ), yn( j ),                 ...
                                  xn(j+1), yn(j+1)  );
                end
            else % Odd columns
                if j == i
                    H(i,j) = 0;
                    G(i,j) =                                        ...
                        findG_par_s(1, xn( i ), yn( i ),            ...
                                       xn( j ), yn( j ),            ...
                                       xn(j+1), yn(j+1),            ...
                                       xn(j+2), yn(j+2),            ...
                                       -1 )                         ...
                        +                                           ...
                        findG_par_s(3, xn( i ), yn( i ),            ...
                                       xn(j-2), yn(j-2),            ...
                                       xn(j-1), yn(j-1),            ...
                                       xn( j ), yn( j ),            ...
                                       +1 );
                else
                    H(i,j) =                                        ...
                        findH(met, 1, xn( i ), yn( i ),             ...
                                      xn( j ), yn( j ),             ...
                                      xn(j+1), yn(j+1),             ...
                                      xn(j+2), yn(j+2)  )           ...
                        +                                           ...
                        findH(met, 3, xn( i ), yn( i ),             ...
                                      xn(j-2), yn(j-2),             ...
                                      xn(j-1), yn(j-1),             ...
                                      xn( j ), yn( j )  );
                    G(i,j) =                                        ...
                        findG(met, 1, xn( i ), yn( i ),             ...
                                      xn( j ), yn( j ),             ...
                                      xn(j+1), yn(j+1),             ...
                                      xn(j+2), yn(j+2)  )           ...
                        +                                           ...
                        findG(met, 3, xn( i ), yn( i ),             ...
                                      xn(j-2), yn(j-2),             ...
                                      xn(j-1), yn(j-1),             ...
                                      xn( j ), yn( j )  );
                end
            end
        end

        % Indirect evaluation of diagonal influence coefficients
        H(i,i) = -sum(H(i,:));
    end
```

1.4 Rearrangement of matrices $[H]$ and $[G]$

Since only one of the two boundary quantities u and u_n is supposed to be known at each node, the resulting system of equations in the form (23) has to be rearranged on the basis of the boundary conditions. If the value of $\{u\}$ is unknown at a node, then the respective column of $[H]$ remains at the left hand side of (23), otherwise this column is multiplied by the known value of $\{u\}$, its sign is switched and it is shifted

to the right hand side of the equation. Similarly, if the value of $\{u_n\}$ is unknown then the respective column of $[G]$ is shifted with opposite sign to the left hand side of the equation. The unknowns are rearranged by examining all the nodes of the boundary. After completing this process, the right hand side of the equation contains only known quantities and, thus, the matrix multiplication results in a single vector

$$[A]\{z\} = \{b\} \ . \tag{26}$$

System (26) is solved for z, which is the vector of unknown boundary values.

find_BEM_unkowns

```
% Matrices re-arrangement.
% Unknowns shifted (and sign changed) to the left-hand side.
% Known quantities shifted (and sign changed) to the right-hand side.
% System solution -> z vector of the unknowns.

A = zeros(I, I);
b = zeros(I, I);

for k = 1:I
    if bc(k) == 0
        % Dirichlet boundary condition
        A(:,k) = -G(:,k);
        b(:,k) = -H(:,k) .* bv(k);
    else
        % Neumann boundary condition
        A(:,k) =  H(:,k);
        b(:,k) =  G(:,k) .* bv(k);
    end
end
```

1.5 Evaluation of line integrals

Matrices $[G]$ and $[H]$ require line integration of functions involving the fundamental solution v of the Laplace operator, or its normal derivative. Two cases can be distinguished, depending on whether node p_i lies inside or outside the element over which the integral si computed.

In the first case we talk about outside integration, and the line integrals are evaluated numerically using a standard Gaussian quadrature:

$$\int_{-1}^{+1} f(\xi)d\xi = \sum_{k=1}^{n} \omega_k f(\xi_k) \ ,$$

where n is the number of integration/Gauss points, whereas ξ_k and ω_k are the abscissas and weights of the Gaussian quadrature of order n. This is the most suitable method for the numerical evaluation of BEM integrals, since it approximates the integral with great accuracy. The accuracy depends not only on the number of integration points, but also on how the integrand varies within the integration interval (a smooth variation of the integrand gives more accurate results).

In the inside integration p_i lies within the element over which the integral is performed; the integration point runs along the element, and at a certain moment it will "hit" p_i: here distance r vanishes and the integrals exhibit a singular behavior, since factors $\ln r$ and $\cos\phi/r$ become infinite. Such singularities are called respectively logarithimic and Cauchy singularity: it can be shown that their respective integrals are still convergent, but an adequate integration technique has to be adopted to obtain the correct value.

Constant BEM

- Outside integration ($i \neq j$)

  In this case point p_i lies outside the j-th element, so $r = |q_j - p_i| \neq 0$ and the integral is regular.

  The numerical evaluation is carried out in the following way:

$$G_{ij} = \int_{-1}^{+1} \frac{1}{2\pi} \ln r(\xi) \frac{l_j}{2} d\xi = \frac{l_j}{4\pi} \sum_{k=1}^{n} \ln r(\xi_k) \omega_k \ . \tag{27}$$

  The integral in (27) can also be evaluated analytically. Referring to Fig. 3 and observing that

$$\mathrm{d}s \cos\phi = r \, \mathrm{d}\alpha \ ,$$

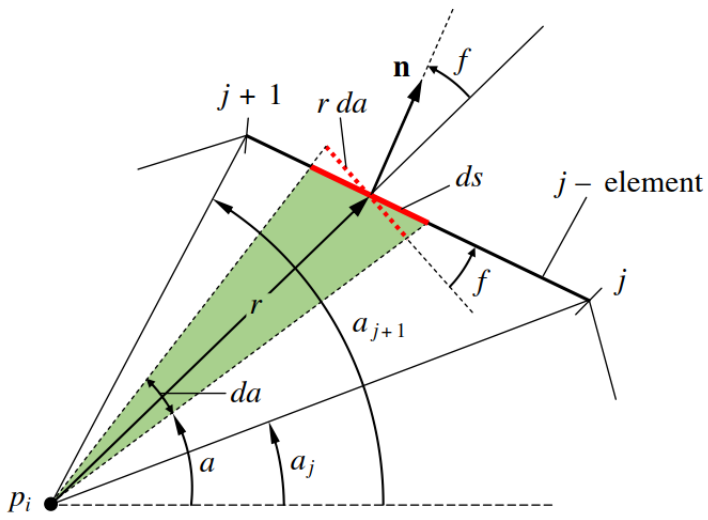

Figure 3: Definition of the angles involved in the numerical integration over constant elements.

we derive

$$\hat{H}_{ij} = \int_{\Gamma_j} \frac{1}{2\pi} \frac{\cos\phi}{r} ds = \int_{\Gamma_j} \frac{1}{2\pi} d\alpha = \frac{\alpha_{j+1} - \alpha_j}{2\pi} \ , \tag{28}$$

where the angles are computed as

$$\tan\alpha_{j+1} = \frac{y_{j+1} - y_i}{x_{j+1} - x_i} \ ,$$
$$\tan\alpha_j = \frac{y_j - y_i}{x_j - x_i} \ .$$

- Inside integration ($i = j$)

  In this case p_i coincides with p_j, and $\mathbf{r}$ lies on the element. Consequently, it is $\phi = \pi/2$ or $\phi = 3\pi/2$, yielding $\cos\phi = 0$ in both cases. Following from

$$x_{pj} = \frac{x_{j+1} + x_j}{2} \ ,$$
$$y_{pj} = \frac{y_{j+1} + y_j}{2} \ ,$$

  we find

$$r(\xi) = \sqrt{\left[x(\xi) - x_{pj}\right]^2 + \left[y(\xi) - y_{pj}\right]^2} = \frac{l_j}{2}|\xi| \ ,$$

  hence the influence coefficients can be computed as

$$\begin{aligned} G_{jj} &= \int_{\Gamma_j} \frac{1}{2\pi} \ln r \, \mathrm{d}s = 2\int_0^{l_j/2} \frac{1}{2\pi} \ln r \, \mathrm{d}r \\ &= \frac{1}{\pi}\left[r\ln r - r\right]_0^{l_j/2} = \frac{1}{\pi}\frac{l_j}{2}\left[\ln\left(\frac{l_j}{2}\right) - 1\right] \ , \end{aligned} \tag{29}$$

$$\begin{aligned} \hat{H}_{jj} &= \frac{1}{2\pi}\int_{\Gamma_j} \frac{\cos\phi}{r} \, \mathrm{d}s = \frac{1}{2\pi}\int_{-1}^{+1} \frac{\cos\phi}{|\xi|} \, \mathrm{d}\xi \\ &= \frac{2}{2\pi}\left[\cos\phi \ln|\xi|\right]_0^1 = 0 \ . \end{aligned} \tag{30}$$

  It has to be highlighted that, for higher order elements, analytical integration is not applicable for the case of Cauchy singularity and, for this reason, other integration techniques are employed.

Linear and Quadratic BEM The case of linear and quadratic BEM must be treated carefully, since over each single element more than one node is defined. In such cases the inside integration has two distinct sub-cases: p and q are in the same element but $p \neq q$ or when p and q are in the same element and $p = q$ [5].

findG

```
% Calculates the G matrix coefficient related to the given set of points.
%
%      met       Geometric element type (1 for linear elements, 2 for
%                quadratic).
%
%       c        Shape function type (2 is allowed only with quadratic
%                elements).
%
%    (xi, yi)    The reference point.
%
%    (x1, y1) (x2, y2) (x3, y2)
%                The boundary points that identify the element. In case of
%                linear elements point (x2, y2) is the middle point between
%                (x1, y1) and (x3, y3).

function [g_c] = findG(met, c, xi, yi, x1, y1, x2, y2, x3, y3)

    f = @(t) shape_fun(met, c, t)                                        ...
             .*                                                          ...
             intG(met, xi, yi, x1, y1, x2, y2, x3, y3, t)                ...
             .*                                                          ...
             abs_J(x1, y1, x2, y2, x3, y3, t);

    g_c = quadgk(f, -1, +1);

end
```

intG

```
% Calculates the kernel (ln r), which is part of the integrand for the
% the coefficients of the G matrix.
%
%      met       Geometric element type (1 for linear elements, 2 for
%                quadratic).
%
%    (xi, yi)    The reference point.
%
%    (x1, y1) (x2, y2) (x3, y2)
%                The boundary points that identify the element. In case of
%                linear elements point (x2, y2) is the middle point between
%                (x1, y1) and (x3, y3).
%
%       t        The point in [-1, 1] where the function has to be
%                evaluated.

function [y] = intG(met, xi, yi, x1, y1, x2, y2, x3, y3, t)

    [rx, ry] = find_R(met, xi, yi, x1, y1, x2, y2, x3, y3, t);

    r = rx + 1i * ry;
    y = (1/(2*pi)) * log(abs(r));

end
```

findH

```
% Calculates the H matrix coefficient related to the given set of points.
%
%      met       Geometric element type (1 for linear elements, 2 for
%                quadratic).
%
%       c        Shape function type (2 is allowed only with quadratic
%                elements).
%
%    (xi, yi)    The reference point.
%
%    (x1, y1) (x2, y2) (x3, y3)
%                The boundary points that identify the element. In case of
%                linear elements point (x2, y2) is the middle point between
%                (x1, y1) and (x3, y3).

function [h_c] = findH(met, c, xi, yi, x1, y1, x2, y2, x3, y3)

    f = @(t) shape_fun(met, c, t)                                          ...
             .*                                                            ...
             intH(met, xi, yi, x1, y1, x2, y2, x3, y3, t)                  ...
             .*                                                            ...
             abs_J(x1, y1, x2, y2, x3, y3, t);

    h_c = quadgk(f, -1, +1);

end
```

intH

```
% Calculates the derivative of the kernel (1/r), which is part of the
% integrand for the coefficients of the H matrix.
%
%      met       Geometric element type (1 for linear elements, 2 for
%                quadratic).
%
%    (xi, yi)    The reference point.
%
%    (x1, y1) (x2, y2) (x3, y3)
%                The boundary points that identify the element. In case of
%                linear elements point (x2, y2) is the middle point between
%                (x1, y1) and (x3, y3).
%
%       t        The point in [-1, 1] where the function has to be
%                evaluated.

function [y] = intH(met, xi, yi, x1, y1, x2, y2, x3, y3, t)

    [rx, ry] = find_R(met, xi, yi, x1, y1, x2, y2, x3, y3, t);
    [nx, ny] = find_N(met, x1, y1, x2, y2, x3, y3, t);

    n = nx + 1i * ny;
    r = rx + 1i * ry;

    y = (1/(2*pi)) * cos(angle(n) - angle(r)) ./ abs(r);

    % theta = angle(n) - angle(r) is the angle between the radial direction
    % and the normal unit vector.

end
```

In the first case, the kernels are singular, i.e. they tend to infinity as point q is approached, but the shape function $\psi_\alpha(\xi)$ in the vicinity of p is of the order $r(p,q)$ and tends to zero. Therefore, the integrals of the product of the kernels and the shape function tend to a finite value instead of being singular and the evaluation can be carried out using the standard Gaussian quadrature.

In the second case the kernels still tend to infinity whereas the shape function tends to unity. This leads to an infinite, or singular, integral of the product.

- Cauchy singularity

  Matrices $[G]$ and $[H]$ are affected only by the boundary geometry, its discretization and the employed type of element. Hence, these matrices do not depend on the boundary conditions. The indirect evaluation of the diagonal influence coefficients H_{ii} for linear and quadratic BEM is based on this property. Considering a closed domain Ω, the diagonal element in the i-th row of matrix $[H]$ is equal to the negative sum of the remaining elements in this row

  $$H_{ii} = -\sum_{\substack{j=1 \\ j\neq i}}^{N} H_{ij} \ .$$

  Therefore, this method avoids the evaluation of any singular integral for the coefficients of matrix $[H]$.

- Logarithmic singularity for linear BEM

  This kind of singularity can be treated analytically by solving the integrals by parts in the local coordinate system, yielding the following analytical expression for the influence coefficients

  $$G_{jj} = g_1^{jj} + g_2^{j(j-1)} \ ,$$

  and the two terms are calculated as [3]

  $$g_1^{jj} = \frac{l_j}{4\pi}\left[\ln(l_j) - 1.5\right] \ ,$$
  $$g_2^{jj-1} = \frac{l_j}{4\pi}\left[\ln(l_{j-1}) - 1.5\right] \ ,$$

  where

  $$l_j = \sqrt{\left(x_{j+1} - x_j\right)^2 + \left(y_{j+1} - y_j\right)^2} \ .$$

findG_lin_s

```
% It has the same goal of findG(), but it handles the logarithmic
% singularity in case of linear elements, which can be integrated
% analitically.

function [g] = findG_lin_s(x1, y1, x2, y2)

    lm = sqrt( (x2 - x1)^2 + (y2 - y1)^2 );

    g = lm/(4*pi) * (log(lm) - 1.5);

end
```

- Logarithmic singularity for quadratic BEM

  This kind of integrals can be computed thanks to the subtraction and series expansion method [6]: it deals with the singularity by subtracting it out, using a function that is found from an appropriate series expansion and which has the same singular properties. This yields to the sum

$$\int_{-1}^{+1} F(\xi)d\xi = \int_{-1}^{+1} F(\xi)^* + \int_{-1}^{+1} \{F(\xi) - F(\xi)^*\}d\xi \ , \tag{31}$$

  where $F(\xi)^*$ has a singularity of the same type as $F(\xi)$, so that the remainder $F(\xi) - F(\xi)^*$ is not singular and its integral can be evaluated using ordinary numerical methods such as the Standard Gaussian quadrature.

  Concerning the construction of the function $F(\xi)^*$, a general procedure is provided by considering the series expansion of $F(\xi)$. As is known, only the integrands appearing in (31) with the form

$$\psi_\alpha(\xi) \ \ln|p(\xi) - p_\alpha| \ |J(\xi)| \ , \tag{32}$$

  need a special treatment. Each of the three terms of (32) can be expanded separately as a Taylor series about the point p_α, where $\alpha = 1, 2$ or 3.

  The latter procedure allows to identify and isolate the singular part of integrand $F(\xi)$, and thus $F(\xi)^*$ can be defined. At this point, the resulting integrals can be evaluated in a similar way to the linear elements case.

findG_par_s

```
% It has the same goal of findG(), but it handles the logarithmic
% singularity in case of quadratic elements. In this case the task
% is performed by approximating the singular integral with a series
% expansion and the subtraction of the singularity. The expansion of
% all the terms (kernel, shape function, jacobian) is performed.

function [g_cS] = findG_par_s(c, xi, yi, x1, y1, x2, y2, x3, y3, t0)

    % Nodes of the element
```

```
    p1 = [x1 y1];
    p2 = [x2 y2];
    p3 = [x3 y3];

    % fd: first derivative
    % sd: second derivative
    % t0: parameter point (-1, 0, +1 correspond to nodes p1, p2, p3)

    p0_fd = (-0.5 + t0) * p1 + (-2*t0) * p2 + (0.5 + t0) * p3;
    p0_sd = p1 - 2*p2 + p3;

    d0 = dot(p0_fd, p0_fd);
    d1 = dot(p0_fd, p0_sd);

    if c == 1
        % p1
        M0_fd = -0.5 + t0;
    elseif c == 2
        % p2
        M0_fd = -2*t0;
    else
        % p3
        M0_fd = 0.5 + t0;
    end

    % Expansion of the complete integrand

    A1 = sqrt(d0) * quadgk(@(t) log(abs(t-t0)), -1, +1);

    A2 = sqrt(d0) * (M0_fd + (d1/d0)) * ...
            quadgk(@(t) (t-t0) .* log(abs(t-t0)), -1, +1);

    B =                                                                 ...
        quadgk(                                                         ...
            @(t) shape_fun(2,c,t)                                       ...
                 .*                                                     ...
                 (2*pi * intG(2, xi, yi, x1, y1, x2, y2, x3, y3, t))    ...
                 .*                                                     ...
                 abs_J(x1, y1, x2, y2, x3, y3, t)                       ...
                 -                                                      ...
                 sqrt(d0) * log(abs(t-t0))                              ...
                 -                                                      ...
                 sqrt(d0) * log(abs(t-t0)) .*                           ...
                    ((t-t0) * (M0_fd + (d1/d0)))  ,                     ...
            -1, +1);

    g_cS = 1/(2*pi) * (A1 + A2 + B);

end
```

1.6 Solution at the internal points of the domain

The solution of (2) yields the unknown boundary quantities u and u_n. Therefore, knowing all the boundary values on Γ, the solution u can be computed at any point

$P(x, y)$ in the domain Ω still by virtue of (2) for $\epsilon(P) = 1$

$$u(P) = \sum_{j=1}^{N} H_{ij} u^j - \sum_{j=1}^{N} G_{ij} u_n^j \ , \tag{33}$$

where the influence coefficients are computed again from the usual integrals, but now p_i is replaced by the internal point P (note that singular integration no longer occurs).

find_BEM_domain

```
% Calculates the value of function u on the domain point (x,y) using the
% given BEM solution.
%
%    met    Geometric element type (1 for linear elements, 2 for quadratic).
%    xn     Nodes' x-coordinates.
%    yn     Nodes' y-coordinates.
%    u      Values of u on the boundary.
%    q      Values of the normal derivative of u on the boundary.
%    x      x-coordinate of the point in which u has to be computed.
%    y      y-coordinate of the point in which u has to be computed.

function [u_dom] = find_BEM_domain(met, xn, yn, u, q, x, y)

    xn(end+1) = xn(1);
    yn(end+1) = yn(1);

    u_dom = 0;

    % Total number of BEM nodes
    I = length(xn) - 1;

    % Construction of G an H vectors and
    % computation of u_dom

    H = zeros(I, 1);
    G = zeros(I, 1);

    if met == 1

        % Linear elements

        xm = zeros(I, 1);
        ym = zeros(I, 1);

        for k = 1:I

            xm(k) = 0.5 * (xn(k) + xn(k+1));
            ym(k) = 0.5 * (yn(k) + yn(k+1));

            xm(I) = 0.5 * (xn(I) + xn(I+1));
            ym(I) = 0.5 * (yn(I) + yn(I+1));

            if k == 1

                H(k) = ...
                    findH(met, 1,    x     ,    y     ,                    ...
                                   xn( 1 ), yn( 1 ),                      ...
```

```
                                xm( 1 ), ym( 1 ),                    ...
                                xn( 2 ), yn( 2 )   )                 ...
                +                                                    ...
                findH(met, 3,   x     ,    y     ,                   ...
                                xn( I ), yn( I ),                    ...
                                xm( I ), ym( I ),                    ...
                                xn( 1 ), yn( 1 )   );
            G(k) =                                                   ...
                findG(met, 1,   x     ,    y     ,                   ...
                                xn( 1 ), yn( 1 ),                    ...
                                xm( 1 ), ym( 1 ),                    ...
                                xn( 2 ), yn( 2 )   )                 ...
                +                                                    ...
                findG(met, 3,   x     ,    y     ,                   ...
                                xn( I ), yn( I ),                    ...
                                xm( I ), ym( I ),                    ...
                                xn( 1 ), yn( 1 )   );

        else

            H(k) =                                                   ...
                findH(met, 1,   x     ,    y     ,                   ...
                                xn( k ), yn( k ),                    ...
                                xm( k ), ym( k ),                    ...
                                xn(k+1), yn(k+1)   )                 ...
                +                                                    ...
                findH(met, 3,   x     ,    y     ,                   ...
                                xn(k-1), yn(k-1),                    ...
                                xm(k-1), ym(k-1),                    ...
                                xn( k ), yn( k )   );
            G(k) =                                                   ...
                findG(met, 1,   x     ,    y     ,                   ...
                                xn( k ), yn( k ),                    ...
                                xm( k ), ym( k ),                    ...
                                xn(k+1), yn(k+1)   )                 ...
                +                                                    ...
                findG(met, 3,   x     ,    y     ,                   ...
                                xn(k-1), yn(k-1),                    ...
                                xm(k-1), ym(k-1),                    ...
                                xn( k ), yn( k )   );

        end

        u_dom = u_dom + H(k) * u(k) - G(k) * q(k);
    end

else

    % Quadratic elements

    for k = 1:I

        if k==1

            H(k) =                                                   ...
                findH(met, 1,   x     ,    y     ,                   ...
                                xn( 1 ), yn( 1 ),                    ...
                                xn( 2 ), yn( 2 ),                    ...
                                xn( 3 ), yn( 3 )   )                 ...
                +                                                    ...
                findH(met, 3,   x     ,    y     ,                   ...
                                xn(I-1), yn(I-1),                    ...
```

```
                                xn( I ), yn( I ),                          ...
                                xn( 1 ), yn( 1 )   );
            G(k) =                                                         ...
                findG(met, 1,   x     ,   y     ,                          ...
                                xn( 1 ), yn( 1 ),                          ...
                                xn( 2 ), yn( 2 ),                          ...
                                xn( 3 ), yn( 3 )   )                       ...
                +                                                          ...
                findG(met, 3,   x     ,   y     ,                          ...
                                xn(I-1), yn(I-1),                          ...
                                xn( I ), yn( I ),                          ...
                                xn( 1 ), yn( 1 )   );

        elseif mod(k,2)==0

            H(k) =                                                         ...
                findH(met, 2,   x     ,   y     ,                          ...
                                xn(k-1), yn(k-1),                          ...
                                xn( k ), yn( k ),                          ...
                                xn(k+1), yn(k+1)   );
            G(k) =                                                         ...
                findG(met, 2,   x     ,   y     ,                          ...
                                xn(k-1), yn(k-1),                          ...
                                xn( k ), yn( k ),                          ...
                                xn(k+1), yn(k+1)   );

        else

            H(k) =                                                         ...
                findH(met, 1,   x     ,   y     ,                          ...
                                xn( k ), yn( k ),                          ...
                                xn(k+1), yn(k+1),                          ...
                                xn(k+2), yn(k+2)   )                       ...
                +                                                          ...
                findH(met, 3,   x     ,   y     ,                          ...
                                xn(k-2), yn(k-2),                          ...
                                xn(k-1), yn(k-1),                          ...
                                xn( k ), yn( k )   );
            G(k) =                                                         ...
                findG(met, 1,   x     ,   y     ,                          ...
                                xn( k ), yn( k ),                          ...
                                xn(k+1), yn(k+1),                          ...
                                xn(k+2), yn(k+2)   )                       ...
                +                                                          ...
                findG(met, 3,   x     ,   y     ,                          ...
                                xn(k-2), yn(k-2),                          ...
                                xn(k-1), yn(k-1),                          ...
                                xn( k ), yn( k )   );
        end

        u_dom = u_dom + H(k) * u(k) - G(k) * q(k);
    end
end

end
```

2 Exercises

We illustrate the solution of two simple exercises: in both cases the analytical solution is known, so that we can compare it with the obtained numerical one.

2.1 Exercise 1: Rectangular Domain

Consider the following problem in the rectangular domain $\Omega = (0,1) \times (0,1)$:

$$\begin{cases} \nabla^2 u = 0 & \text{in } \Omega \\ u = 0 & \text{on } x = 0 \text{ and for } y \in (0,1) \\ u = \cos(\pi y) & \text{on } x = 1 \text{ and for } y \in (0,1) \\ \dfrac{\partial u}{\partial n} = 0 & \text{on } y \in \{0,1\} \text{ and for } x \in (0,1) \end{cases} . \tag{34}$$

It can be verified that the exact solution is

$$u(x,y) = \frac{\sinh(\pi x)\cos(\pi y)}{\sinh(\pi)}. \tag{35}$$

To approach the problem numerically we proceed by steps. The first one is to define the nodes to discretize the boundary.

Exercise1

```
% Step 1: define the boundary nodes.

% Number of nodes per side (the boundary is a square), excluding corners.
n_side = 10;

x_range = linspace(0, 1, n_side+2).';
y_range = linspace(0, 1, n_side+2).';

% Remove the extremes so to avoid corners.
x_range([1, end]) = [];
y_range([1, end]) = [];

nodes = [
    x_range             zeros(n_side, 1)   % Bottom side
    ones(n_side, 1)     y_range            % Right side
    flip(x_range)       ones(n_side, 1)    % Top side
    zeros(n_side, 1)    flip(y_range)      % Left side
];
```

Note that corners have been avoided in the above code: in general it is not easy to deal with them, plus in this exercise we can manage to achieve good accuracy even by ignoring them.

The second step requires to define the boundary conditions. According to the specifications of function `solve_BEM_unkowns` we need to define two vectors:

- `boundary_conditions` tells for each node whether it is assigned to a Dirichlet boundary condition (0) or a Neumann boundary condition (1);
- `boundary_values` holds the actual nodal values from the boundary conditions.

Exercise1

```
% Step 2 : define the boundary conditions.

% First we create a vector that tells for each node whether it is assigned
% to a Dirichlet boundary conditions (0) or a Neumann boundary condition
% (1).

boundary_conditions = [
    ones(n_side, 1)   % Bottom side: Neumann
    zeros(n_side, 1)  % Right side:  Dirichlet
    ones(n_side, 1)   % Top side:    Neumann
    zeros(n_side, 1)  % Left side:   Dirichlet
];

% Then we assign the proper boundary values to each node.
boundary_values = [
    zeros(n_side, 1)   % Du = 0
    cos(pi * y_range)  % u = cos(pi*y)
    zeros(n_side, 1)   % Du = 0
    zeros(n_side, 1)   % u = 0
];
```

The third step is to solve the constituted BEM problem and to build the final boundary solution vectors `u` (for u) and `Du` (for $\partial_n u$). We can perform these tasks in one dedicated function.

solve_BEM_problem

```
% Calculates the BEM solution for the problem described by the given
% parameters.

function [u, Du] = ...
    solve_BEM_problem(nodes, boundary_conditions, boundary_values, order)

    z = find_BEM_unknowns(order, nodes(:, 1), nodes(:, 2), ...
                          boundary_conditions, boundary_values);

    N = numel(boundary_values);

    u = zeros(N,1);
    Du = zeros(N,1);

    boundary_conditions = logical(boundary_conditions);

    u(~boundary_conditions) = boundary_values(~boundary_conditions);
    u( boundary_conditions) = z(boundary_conditions);

    Du( boundary_conditions) = boundary_values(boundary_conditions);
    Du(~boundary_conditions) = z(~boundary_conditions);

end
```

Exercise1

```
% Step 3: solve the boundary problem.

% Order of the adopted elements (1 for linear, 2 for quadratic).
order = 1;

[u, Du] = ...
    solve_BEM_problem(nodes, boundary_conditions, boundary_values, order);
```

We can now compare the obtained numerical solution with the exact one.

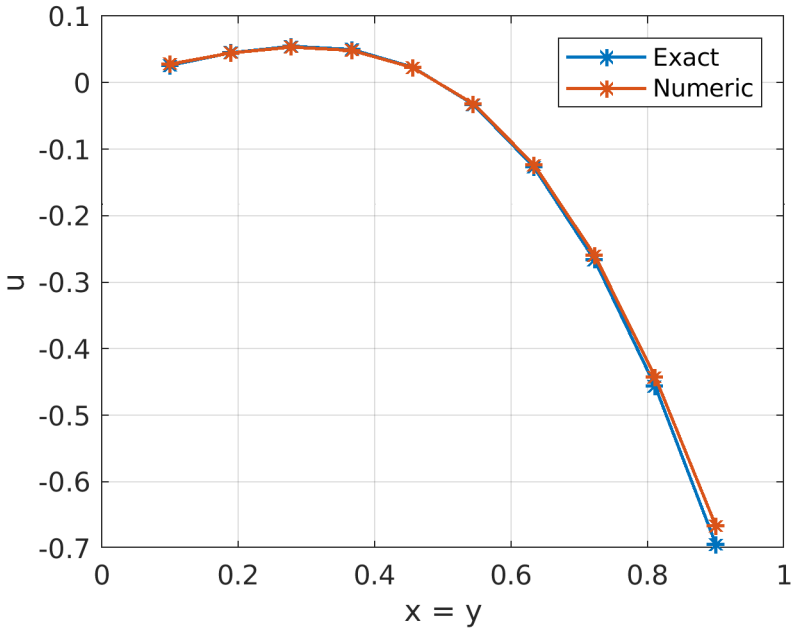

Figure 4: Comparison between exact and numerical solution (linear elements) on a diagonal line $y = x$.

An accurate evaluation of the solution accuracy is performed through a convergence test. Several simulations of the same problem are run, with an increasing number of nodes. Then, the error of each simulation is computed and its trend against the nodes' number is analyzed. Here we adopt the following error norm:

$$\text{Error}(u) = \max_{(x,y)\in\mathcal{T}} \left| \frac{u^{\text{exact}}(x,y) - u^{\text{numeric}}(x,y)}{u^{\text{exact}}(x,y)} \right|, \tag{36}$$

where $\mathcal{T}$ is the set of test points on which u^{exact} and u^{numeric} are evaluated.

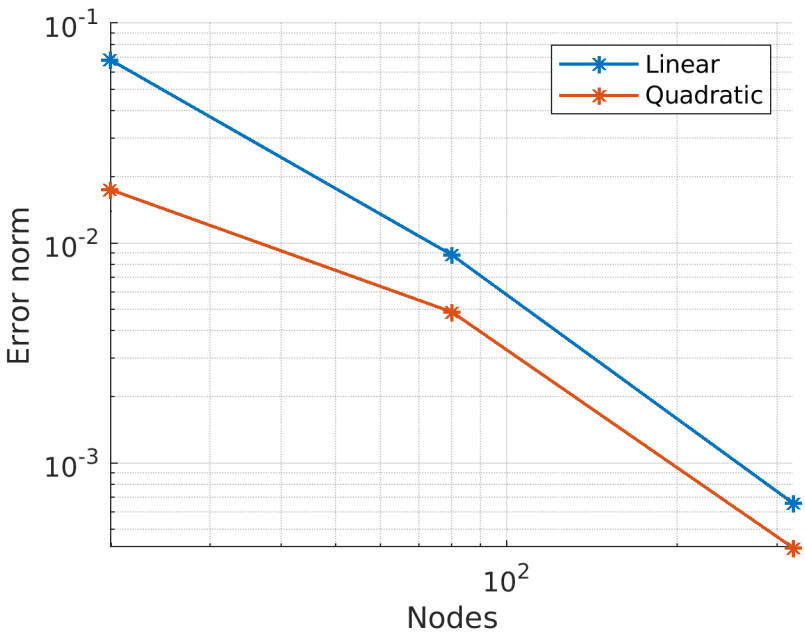

Figure 5: Convergence test with 20, 40 and 320 boundary nodes; the test points' set is $\mathcal{T} = \left\{(x, y) : y = x = 0.1\left(1 + \frac{8}{9}k\right),\ k = 0, \ldots, 9\right\}$. As one might expect, the solution obtained with quadratic elements is more accurate than the one obtained with linear elements. On the other hand, as the number of nodes increases the discrepancy of the results shrinks: the domain boundary indeed is rectangular, hence there is no substantial difference among using straight or curvilinear elements here.

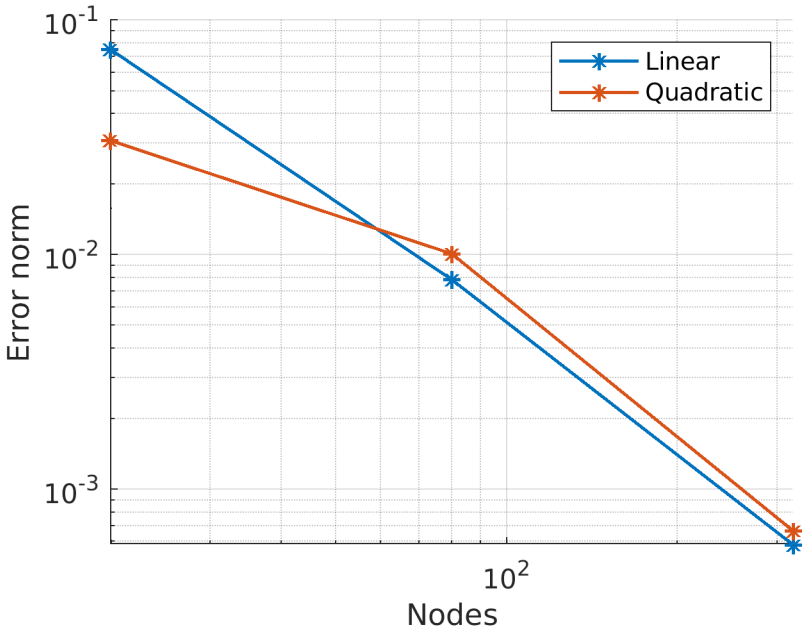

Figure 6: This is another convergence test with 20, 40, 320 nodes and $\mathcal{T} = \left\{(x, y) : x = 0.8, y = 0.1\,(0.5 + k)\,,\ k = 0, \ldots, 9\right\}$. With respect to the previous case, here the test points are much closer to the boundary; this may lead to lower accuracy because of the involved quasi-singular integrations, resulting in unusual error trends like the depicted one.

2.2 Exercise 2: Quarter-Circle Domain

The second exercise is set in a quarter-circle domain $\Omega = \{(x, y) : x > 0, y > 0, x^2 + y^2 < 1\}$:

$$\begin{cases} \nabla^2 u = 0 & \text{in } \Omega \\ u = y & \text{on } x = 0 \text{ and for } y \in (0, 1) \\ u = x + y & \text{on } x^2 + y^2 = 1 \text{ with } x > 0, y > 0 \\ \dfrac{\partial u}{\partial n} = -1 & \text{on } y = 0 \text{ and for } x \in (0, 1) \end{cases} . \tag{37}$$

The exact solution is

$$u(x, y) = x + y. \tag{38}$$

The formal steps for solving the problem are the same of Exercise 1, hence we can directly write the solution code.

Exercise2

```
% Step 1: define the boundary nodes.

% Number of nodes per segment (excluding corners).
n_segments   = 15;
% Number of nodes in the circle arc portion (excluding corners).
n_circle_arc = 10;

segments_ranges = linspace(0, 1, n_segments+2).';
circle_angles   = linspace(0, pi/2, n_circle_arc+2).';

% Remove the extremes so to avoid corners.
segments_ranges([1, end]) = [];
circle_angles([1, end])   = [];

nodes = [
    segments_ranges          zeros(n_segments, 1)
    cos(circle_angles)       sin(circle_angles)
    zeros(n_segments, 1)     flip(segments_ranges)
];

% Step 2: define the boundary conditions.

% First we create a vector that tells for each node whether it is assigned
% to a Dirichlet boundary conditions (0) or a Neumann boundary condition
% (1).

boundary_conditions = [
    ones(n_segments, 1)       % Bottom segment: Neumann
    zeros(n_circle_arc, 1)    % Circular arc:   Dirichlet
    zeros(n_segments, 1)      % Left segment:   Dirichlet
];

% Then we assign the proper boundary values to each node.

boundary_values = [
    -1 * ones(n_segments, 1)
```

```
    cos(circle_angles) + sin(circle_angles)
    flip(segments_ranges)
];

% Step 3: solve the boundary problem.

% Order of the adopted elements (1 for linear, 2 for quadratic).
order = 1;

[u, Du] = ...
    solve_BEM_problem(nodes, boundary_conditions, boundary_values, order);
```

Again, we can compare the numerical solution with the exact one and run a convergence test to evaluate the accuracy.

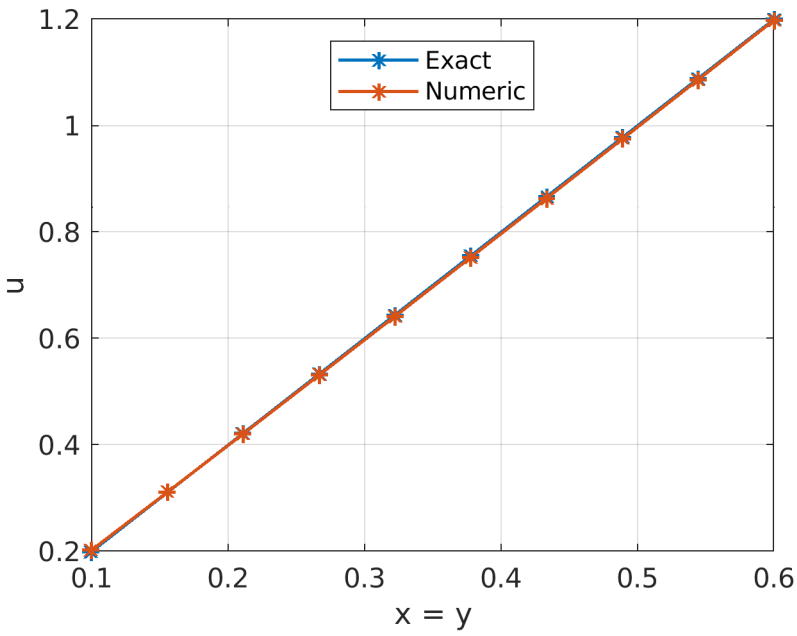

Figure 7: Comparison between exact and numerical solution (linear elements) on a diagonal line $y = x$.

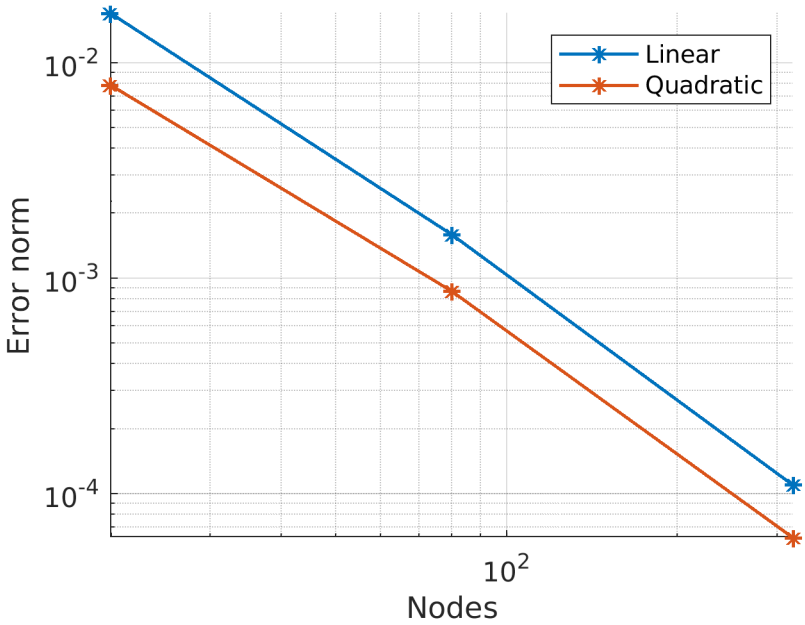

Figure 8: Convergence test with 20, 40 and 320 boundary nodes; the test points' set is $\mathcal{T} = \left\{(x, y) : y = x = 0.1\left(1 + \frac{8}{9}k\right),\ k = 0, \ldots, 9\right\}$. Again the solution obtained with quadratic elements is more accurate than the one obtained with linear elements, as expected. In this case there is a systematic advantage in using curvilinear elements because of the non-straight boundary geometry.

References

[1] L. Biccioli. BEM Reconstruction of 2D Magnetic Field Maps from Measured Boundary Data in Accelerator Magnets. Master's thesis, Politecnico di Milano, Italy, 2018.

[2] M. Costabel. Principles of Boundary Element Methods. Technische Hochschule Darmstadt.

[3] J.T. Katsikadelis. The Boundary Element Method for Engineers and Scientists. ELSEVIER, 2016.

[4] A. Keng-Cheng. Introducing the boundary element method with matlab. International Journal of Mathematical Education in Science and Technology, 2008.

[5] G. Beer, I. Smith, and C. Duenser. The Boundary Element Method with Programming. Springer, 2008.

[6] W.S. Hall. The Boundary Element Method. Springer, 1994.